

# Federated LoRA Fine-Tuning of Language Models for Network Intrusion Detection on NF-ToN-IoT

Interim report · 17 September 2026 · all 10 runs complete

## Experiment

|          |                                         |
|----------|-----------------------------------------|
| Project  | fed-llm-ids                             |
| Tracking | comet.com / text-films / fed-llm-ids    |
| Code     | github.com/asfi50/Fed_GNN (branch: llm) |
| Adapters | huggingface.co / fed-ids-<model-name>   |

## Environment

|           |                                                            |
|-----------|------------------------------------------------------------|
| Hardware  | Kaggle notebook · NVIDIA Tesla T4, 16 GB                   |
| Software  | transformers 5.17.0 · peft 0.21.0 · torch 2.10.0+cu128     |
| Precision | bert, t5: fp32 · llama, qwen: fp16 base with fp32 adapters |
| Seed      | 42 (partitioning, sampling and initialisation)             |

## Models

| Checkpoint                | Family          | Parameters | Trainable (LoRA) | Share   |
|---------------------------|-----------------|------------|------------------|---------|
| ■ bert-base-uncased       | encoder         | 110 M      | 0.60 M           | 0.543 % |
| ■ t5-base                 | encoder-decoder | 225 M      | 1.77 M           | 0.785 % |
| ■ meta-llama/Llama-3.2-1B | decoder         | 1,239 M    | 3.43 M           | 0.277 % |
| ■ Qwen/Qwen3.5-2B         | decoder         | 2,215 M    | 1.50 M           | 0.068 % |
| ■ google/gemma-3-1b-pt    | decoder         | 1,003 M    | 2.99 M           | 0.298 % |

## Dataset

|               |                                                  |
|---------------|--------------------------------------------------|
| Source        | NF-ToN-IoT · netflow records · 10 attack classes |
| Rows          | 1,379,274 (used in full, no subsampling)         |
| Validation    | 137,928 stratified, shared by every run          |
| Test          | 137,928 stratified, shared by every run          |
| Training pool | 1,103,418 partitioned across 5 clients           |

## Input representation

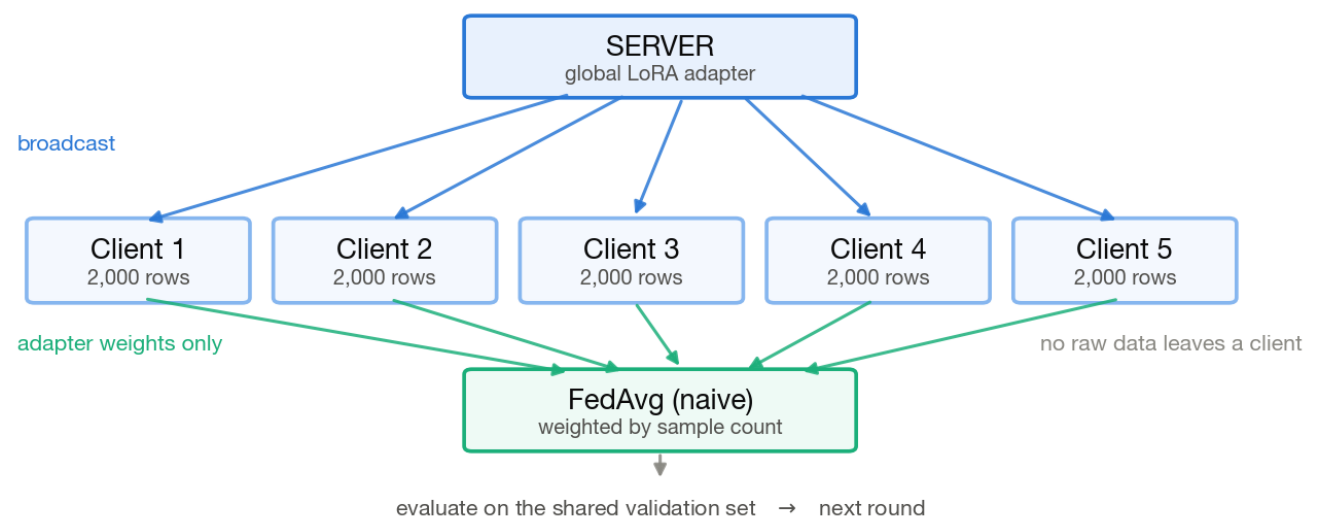
Each netflow row is serialised into a compact **key=value** string — protocol, ports, byte and packet counts, TCP flags and duration — then tokenised to at most 128 tokens.

**Label** (the binary attack flag) and **Attack** (the target) are dropped as outright leaks, and both IP columns with them: attacker addresses are near-unique per class in this dataset, so keeping them would let a model memorise addresses rather than learn traffic behaviour.

*Scope.* Naive FedAvg aggregation only. The three alternative strategies implemented in the pipeline — delta-SVD, FFA-LoRA and performance weighting — are left for a separate study.

# Pipeline

One federated round, repeated 10 times.



## Federated configuration

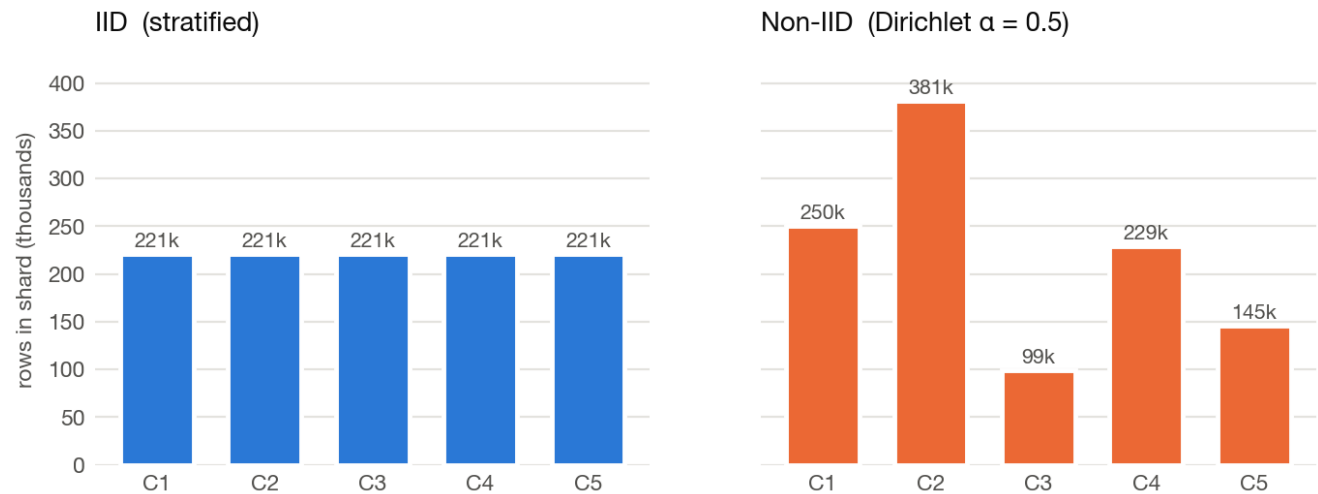
| Setting                   | Value                                            |
|---------------------------|--------------------------------------------------|
| Clients                   | 5                                                |
| Rounds                    | 10                                               |
| Local epochs              | 1 per round                                      |
| Rows per client per round | 2,000 — a fresh slice each round, never repeated |
| Rows seen per client      | 20,000 of its own shard                          |
| Aggregation               | naive FedAvg, weighted by client sample count    |

## Adaptation

| Setting        | Value                                                      |
|----------------|------------------------------------------------------------|
| Method         | LoRA · rank 16 · alpha 32 · dropout 0.05                   |
| Target modules | query, value (bert) · q, v (t5) · q, k, v, o (llama, qwen) |
| Optimiser      | AdamW · lr 2e-4 · weight decay 0.01 · 6 % warmup           |
| Batch size     | 32 (bert, t5) · 16 (llama, qwen)                           |
| Class weights  | inverse frequency, clipped at 50                           |

## Client shards

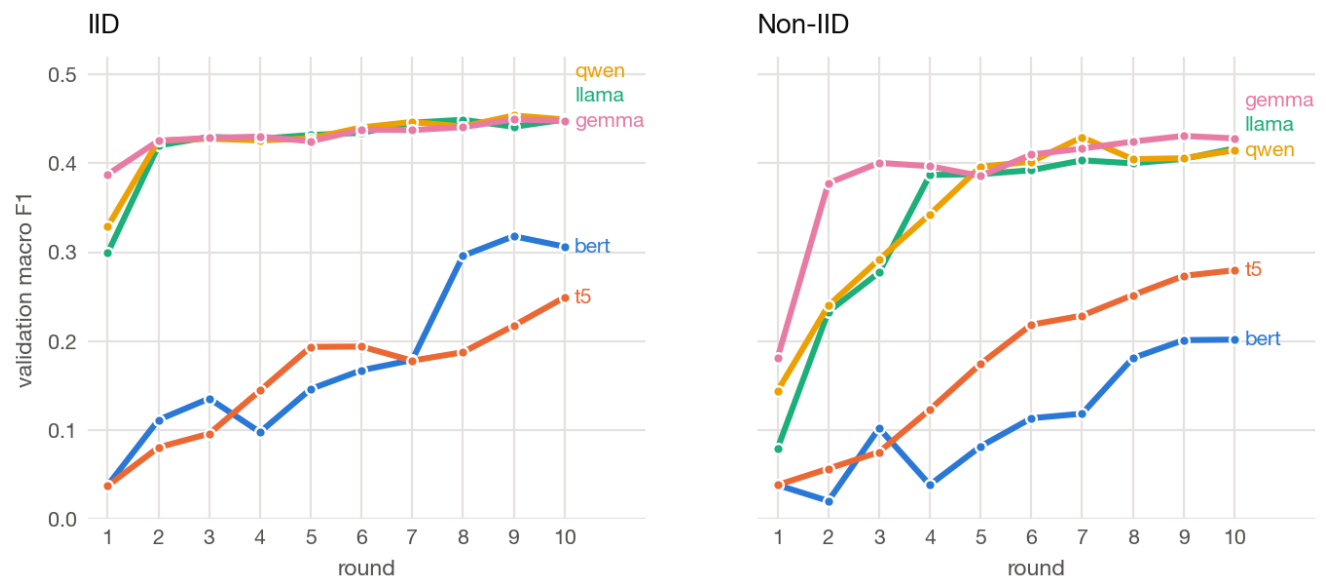
How the 1,103,418 training rows are divided between the five clients.



Each client owns a fixed private shard and walks through it in consecutive slices, one per round, so it never trains on the same row twice. The per-round sample keeps the client's own class distribution intact — rebalancing it would erase the very skew the non-IID split exists to measure. Largest-to-smallest shard ratio:  $1.0\times$  IID,  $3.9\times$  non-IID.

## Convergence

Validation macro F1 after each aggregation round.



Every model is still climbing at round 10 — none has converged. The decoder models pull away within two rounds and hold the gap; bert and t5 are still below 0.3 when the budget runs out.

## Results

Held-out test set · 30,000 stratified rows · identical for every run.

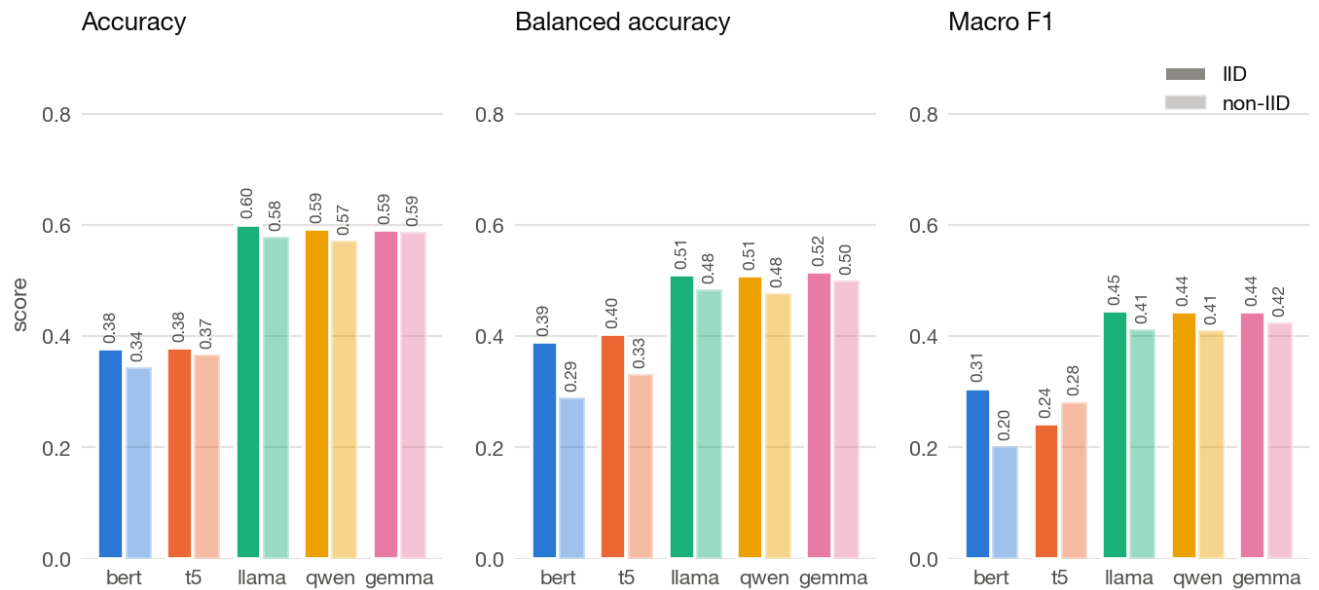
### IID partition

| Model | Accuracy | Balanced acc. | Macro F1 | Macro F1 *   | Wall clock |
|-------|----------|---------------|----------|--------------|------------|
| bert  | 0.378    | 0.389         | 0.307    | <b>0.383</b> | 20 min     |
| t5    | 0.380    | 0.403         | 0.243    | <b>0.303</b> | 35 min     |
| llama | 0.601    | 0.510         | 0.446    | <b>0.558</b> | 32 min     |
| qwen  | 0.593    | 0.508         | 0.444    | <b>0.555</b> | 88 min     |
| gemma | 0.591    | 0.516         | 0.444    | <b>0.555</b> | 174 min    |

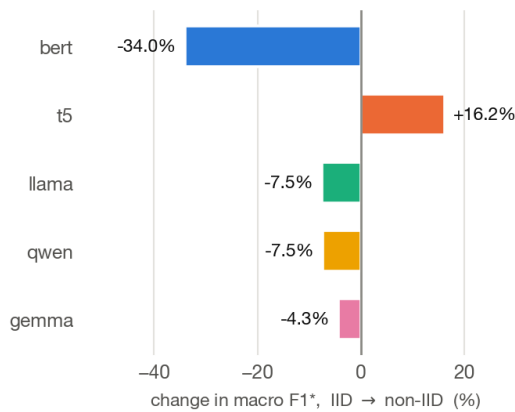
### Non-IID partition (Dirichlet $\alpha = 0.5$ )

| Model | Accuracy | Balanced acc. | Macro F1 | Macro F1 *   | Wall clock |
|-------|----------|---------------|----------|--------------|------------|
| bert  | 0.344    | 0.291         | 0.202    | <b>0.253</b> | 20 min     |
| t5    | 0.368    | 0.333         | 0.282    | <b>0.353</b> | 35 min     |
| llama | 0.579    | 0.484         | 0.413    | <b>0.516</b> | 31 min     |
| qwen  | 0.572    | 0.478         | 0.411    | <b>0.513</b> | 88 min     |
| gemma | 0.588    | 0.501         | 0.425    | <b>0.531</b> | 173 min    |

\* Macro F1 recomputed over the 8 classes excluding mitm and ransomware — see page 5. Wall clock is not comparable across models: gemma carries a 262 k-token vocabulary, so its embedding and output layers dominate a 1 B budget and it runs 5× slower than llama at a smaller parameter count.



## Cost of heterogeneity



All three decoders land on the same number: **0.558, 0.555, 0.555** on IID. They come from three vendors and span 1.0 B to 2.2 B parameters, yet none pulls ahead — for this task, performance saturates around 1 B.

Where they differ is under heterogeneity. **gemma gives up only 4.3 %** moving to the non-IID split, llama and qwen **7.5 %**, and bert **34 %**. The architecture that ties on IID is the one that holds up best when clients stop looking alike. t5 moves the other way, but its scores are weak and unstable throughout, so the sign is not meaningful.

## Per-class performance

F1 on the held-out test set, all models and both partitions.

|                 | Benign | backdoor | ddos | dos  | injection | mitm | password | ransomware | scanning | xss  |
|-----------------|--------|----------|------|------|-----------|------|----------|------------|----------|------|
| bert · IID      | 0.60   | 0.93     | 0.52 | 0.27 | 0.35      | 0.00 | 0.19     | 0.00       | 0.00     | 0.21 |
| t5 · IID        | 0.50   | 0.34     | 0.59 | 0.30 | 0.37      | 0.00 | 0.07     | 0.00       | 0.00     | 0.26 |
| llama · IID     | 0.99   | 0.99     | 0.74 | 0.53 | 0.49      | 0.00 | 0.42     | 0.00       | 0.01     | 0.28 |
| qwen · IID      | 0.99   | 0.99     | 0.74 | 0.54 | 0.48      | 0.00 | 0.41     | 0.00       | 0.00     | 0.29 |
| gemma · IID     | 0.99   | 0.99     | 0.75 | 0.53 | 0.45      | 0.00 | 0.42     | 0.00       | 0.01     | 0.30 |
| bert · non-IID  | 0.68   | 0.00     | 0.54 | 0.39 | 0.00      | 0.00 | 0.28     | 0.00       | 0.00     | 0.13 |
| t5 · non-IID    | 0.73   | 0.97     | 0.55 | 0.12 | 0.07      | 0.00 | 0.14     | 0.00       | 0.00     | 0.24 |
| llama · non-IID | 0.99   | 0.99     | 0.73 | 0.45 | 0.43      | 0.00 | 0.34     | 0.00       | 0.00     | 0.19 |
| qwen · non-IID  | 0.98   | 0.99     | 0.71 | 0.51 | 0.41      | 0.00 | 0.38     | 0.00       | 0.00     | 0.12 |
| gemma · non-IID | 0.99   | 0.99     | 0.74 | 0.45 | 0.45      | 0.00 | 0.39     | 0.00       | 0.00     | 0.23 |

## Two different failure modes

| Class      | Rows in dataset | Share    | Rows seen / client / round | F1 (llama) |
|------------|-----------------|----------|----------------------------|------------|
| ransomware | 142             | 0.010 %  | 0.2                        | 0.00       |
| mitm       | 1,295           | 0.094 %  | 1.9                        | 0.00       |
| scanning   | 21,467          | 1.556 %  | 31.1                       | 0.01       |
| backdoor   | 17,247          | 1.250 %  | 25.0                       | 0.99       |
| ddos       | 326,345         | 23.661 % | 473.2                      | 0.74       |

Class size does not explain the failures. The three *smallest* classes are ransomware, mitm and **backdoor** — and backdoor scores **0.99**, better than injection, which has 27× more data. Two separate problems are at work.

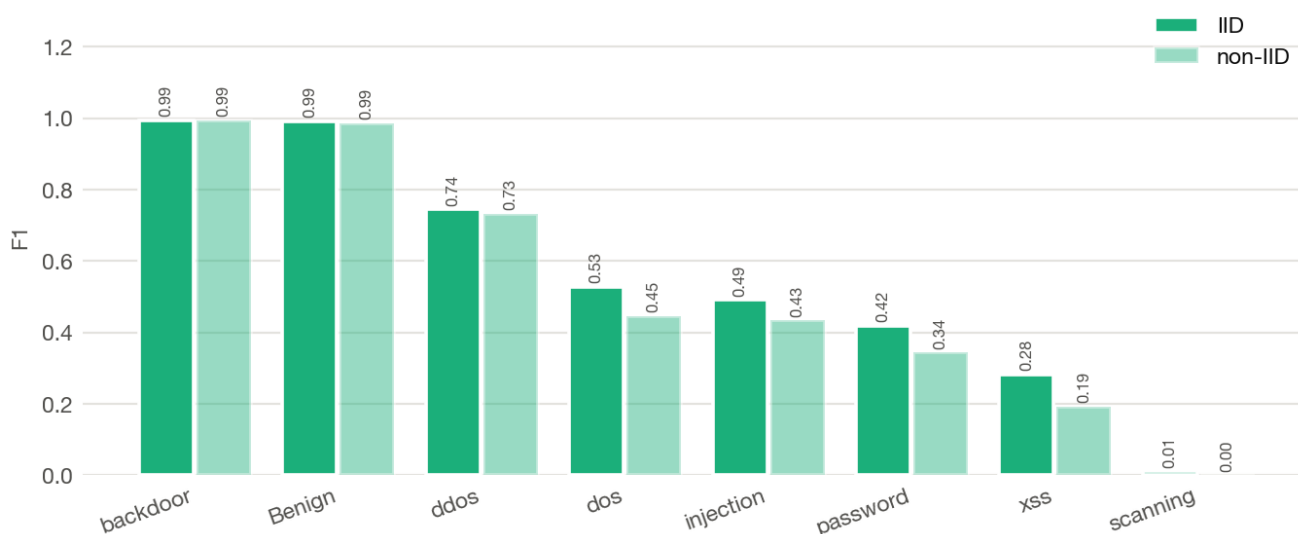
**1. Starvation** — **ransomware, mitm**. A client draws 2,000 rows per round, so it meets a ransomware row roughly once every ten rounds. The test set holds only 3 ransomware and 28 mitm rows; no claim can be made either way. The GNN branch already excludes both, and matching that here would make the branches comparable.

**2. Inseparability** — **scanning, injection, xss, password**. scanning has 467 test rows and ~31 per client per round, which is ample, yet it is predicted only 68 times across the whole test set. These four web-layer attacks collapse into each other, and the table below shows where each one actually lands.

| True class   | Where its predictions go (llama, IID)                               |
|--------------|---------------------------------------------------------------------|
| ● scanning   | password 52 % · xss 19 % · ddos 15 % · dos 11 % · <b>itself 1 %</b> |
| ● injection  | password 38 % · <b>itself 34 %</b> · xss 26 %                       |
| ● xss        | <b>itself 47 %</b> · password 46 % · injection 6 %                  |
| ● mitm       | xss 39 % · Benign 29 % · ddos 11 % · injection 11 %                 |
| ● ransomware | Benign 100 % (3 test rows)                                          |

The serialised flow carries ten fields — ports, protocol, byte and packet counts, TCP flags, duration. That is enough to separate ddos from Benign, but evidently not enough to tell one web-layer attack from another; the model routes most of them into a single password-shaped cluster. This is a limit of the features, not of the federation or the training budget, and it is the most likely reason accuracy stalls near 0.60.

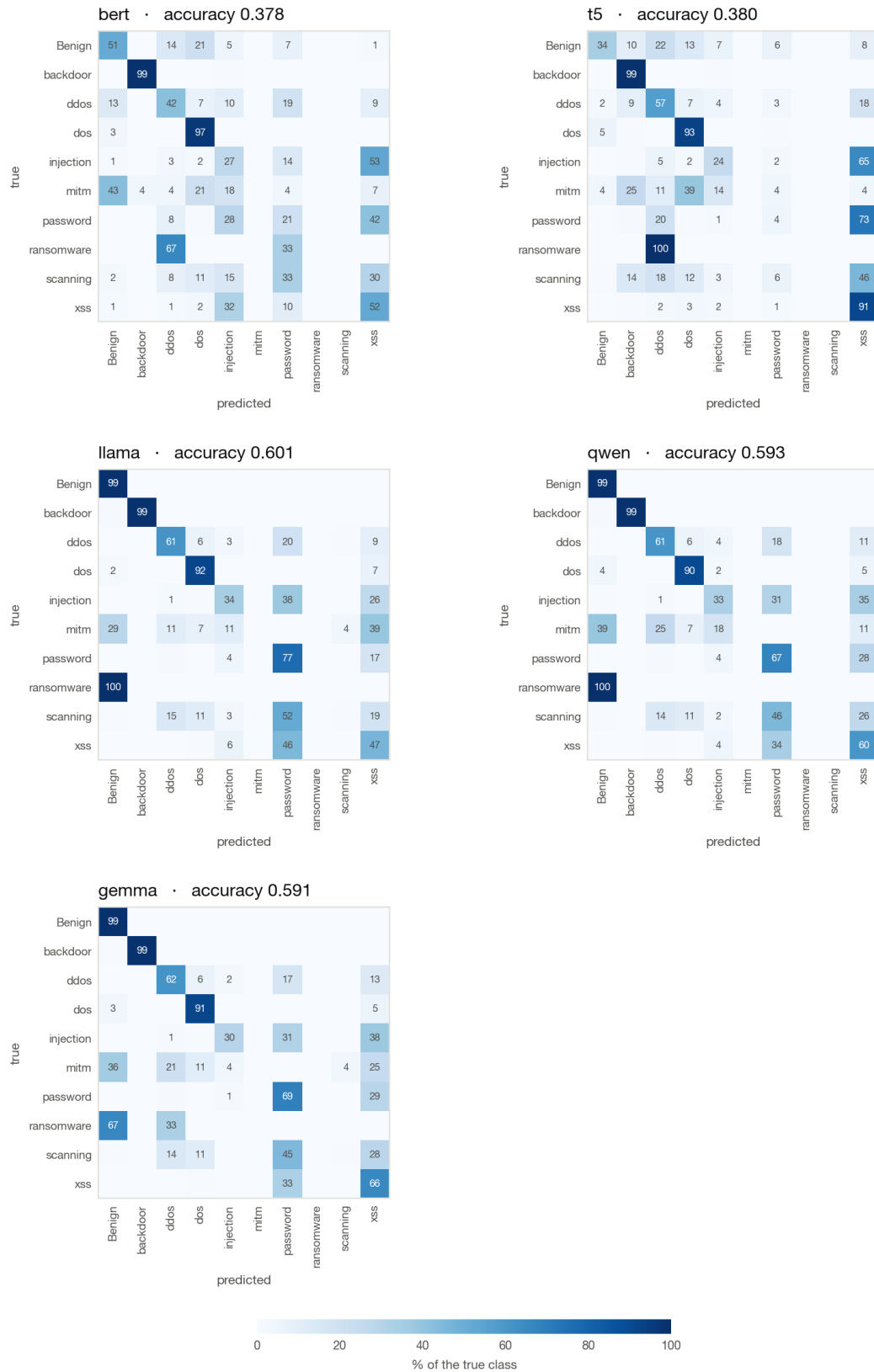
## Where the strongest model still struggles



llama-3.2-1B. Benign and backdoor are essentially solved; everything below ddos degrades, and the non-IID partition costs the most on the classes that were already weakest.

## Confusion matrices — IID partition

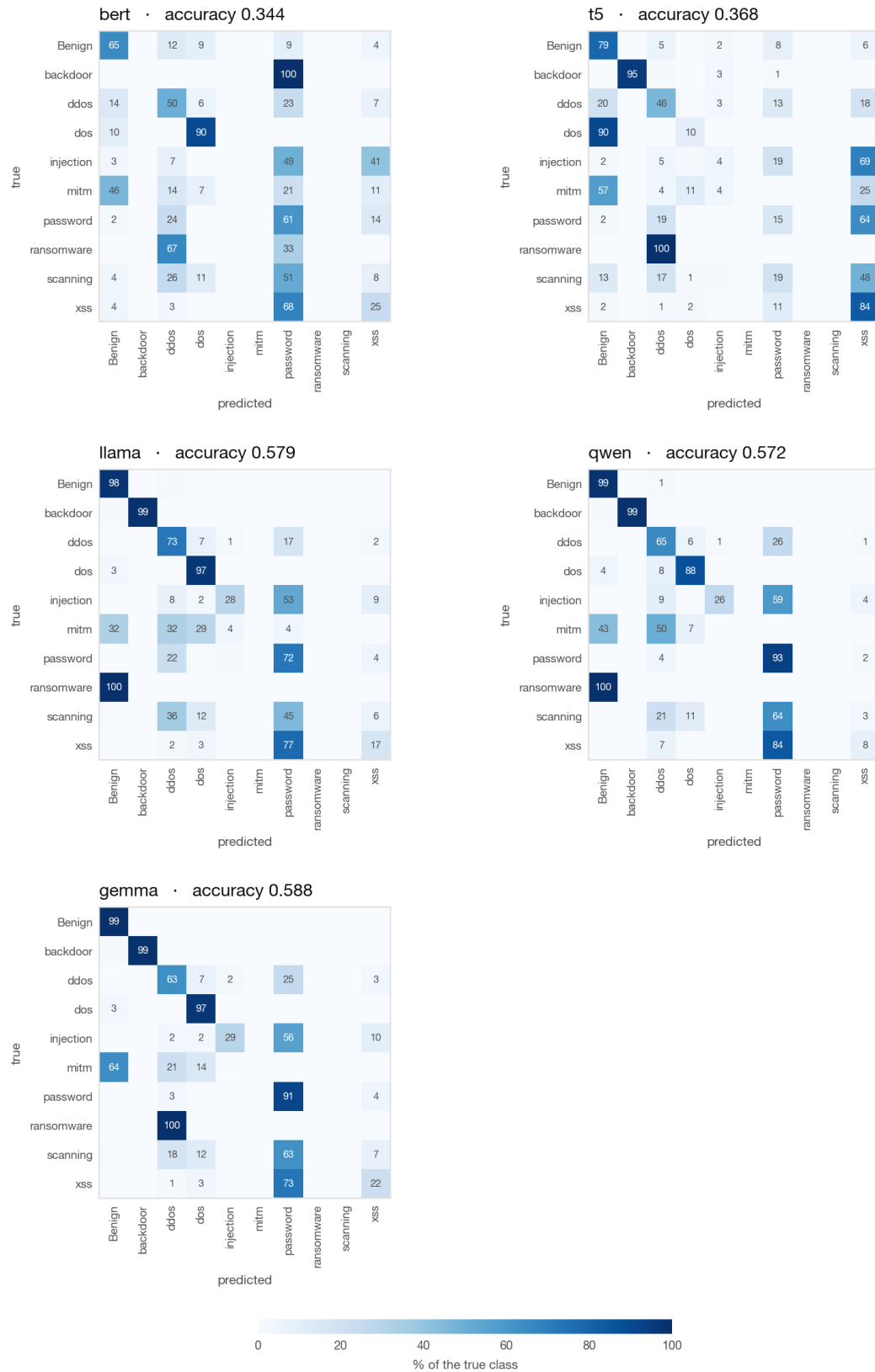
Rows are true classes, columns predicted. Each cell is the percentage of that true class sent to that column, so every row sums to 100.



A dark diagonal means predictions land on the correct class. The decoder models hold a clear diagonal for Benign, backdoor and ddos, while the lower rows stay pale because those classes are rarely predicted at all.

## Confusion matrices — Non-IID partition

Rows are true classes, columns predicted. Each cell is the percentage of that true class sent to that column, so every row sums to 100.



A dark diagonal means predictions land on the correct class. The decoder models hold a clear diagonal for Benign, backdoor and ddos, while the lower rows stay pale because those classes are rarely predicted at all.

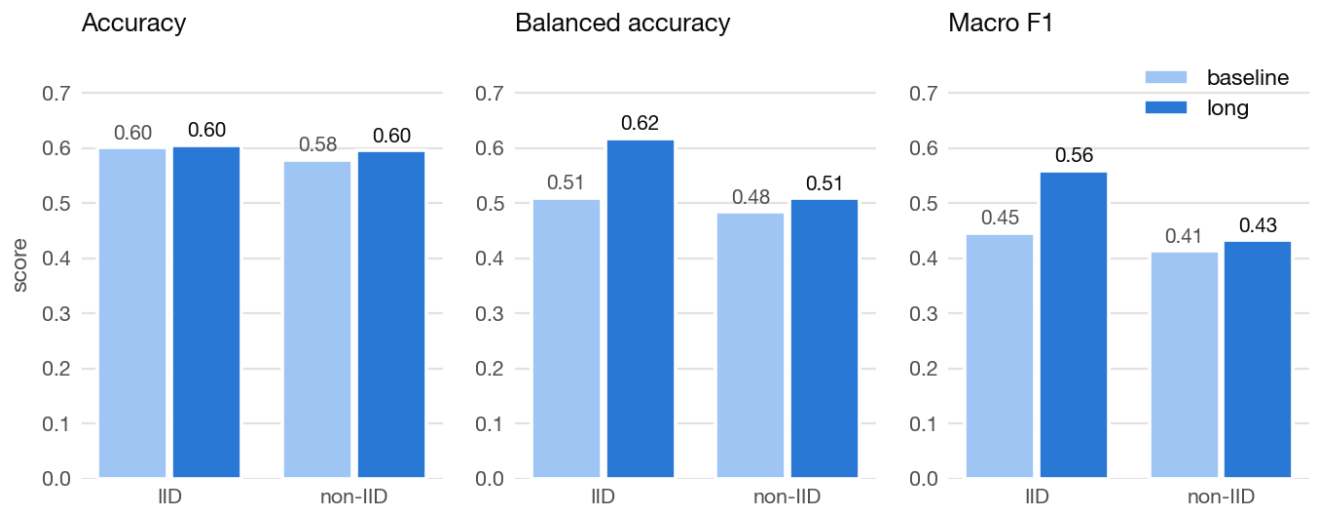
## Extended run — llama at 4.8x the budget

llama stopped improving early in the main study: on IID it gained only 0.029 macro F1 between round 2 and round 10, and the training loss stopped falling after round 3. This run gives the same model a larger budget to separate a compute limit from a real ceiling. Nothing else changes — learning rate, LoRA rank, batch size and aggregation are all as before.

| Setting                | Main study | Extended run |
|------------------------|------------|--------------|
| Rows / client / round  | 2,000      | 8,000        |
| Rounds                 | 10         | 12           |
| Steps / client / round | 125        | 500          |
| Total steps / client   | 1,250      | 6,000        |
| Total rows / client    | 20,000     | 96,000       |
| Wall clock             | ~32 min    | ~1.9 h       |

96,000 is bounded by the smallest non-IID shard (98,616 rows), so every client still walks through fresh rows rather than wrapping back to the start.

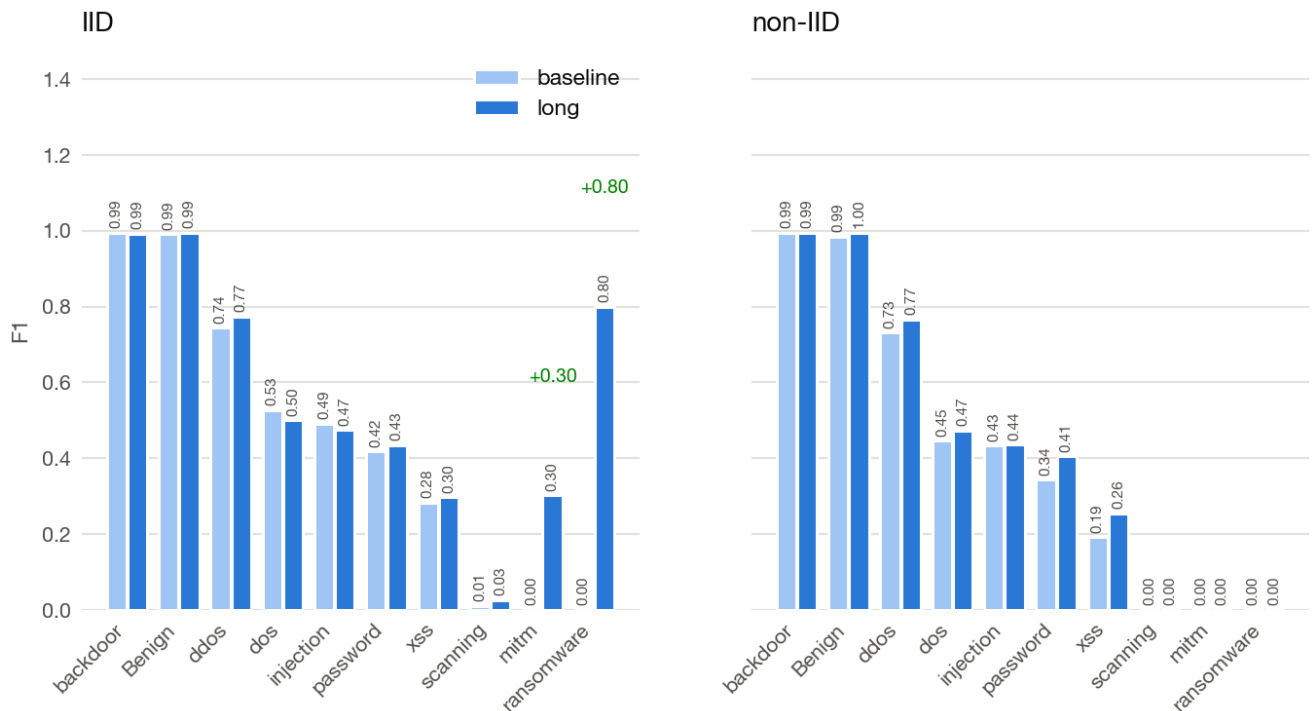
### What changed



| Partition | Accuracy             | Balanced acc.        | Macro F1             | Macro F1 *           |
|-----------|----------------------|----------------------|----------------------|----------------------|
| IID       | 0.601 → <b>0.605</b> | 0.510 → <b>0.618</b> | 0.446 → <b>0.559</b> | 0.558 → <b>0.562</b> |
| non-IID   | 0.579 → <b>0.596</b> | 0.484 → <b>0.509</b> | 0.413 → <b>0.433</b> | 0.516 → <b>0.541</b> |

The two macro F1 columns disagree, and the per-class view below shows why. Over the eight well-populated classes the IID run is flat (+0.7 %) while non-IID gains 4.8 %, which narrows the heterogeneity penalty from -7.5 % to -3.7 %.

## Per-class F1, before and after



Baseline in light blue, extended run in dark. Only bars gaining more than 0.15 are annotated.

On IID, mitm rises from 0.000 to 0.302 and ransomware from 0.000 to 0.800. On non-IID both stay at exactly 0.000. The same model, the same budget and the same test set produce opposite outcomes on these two classes.

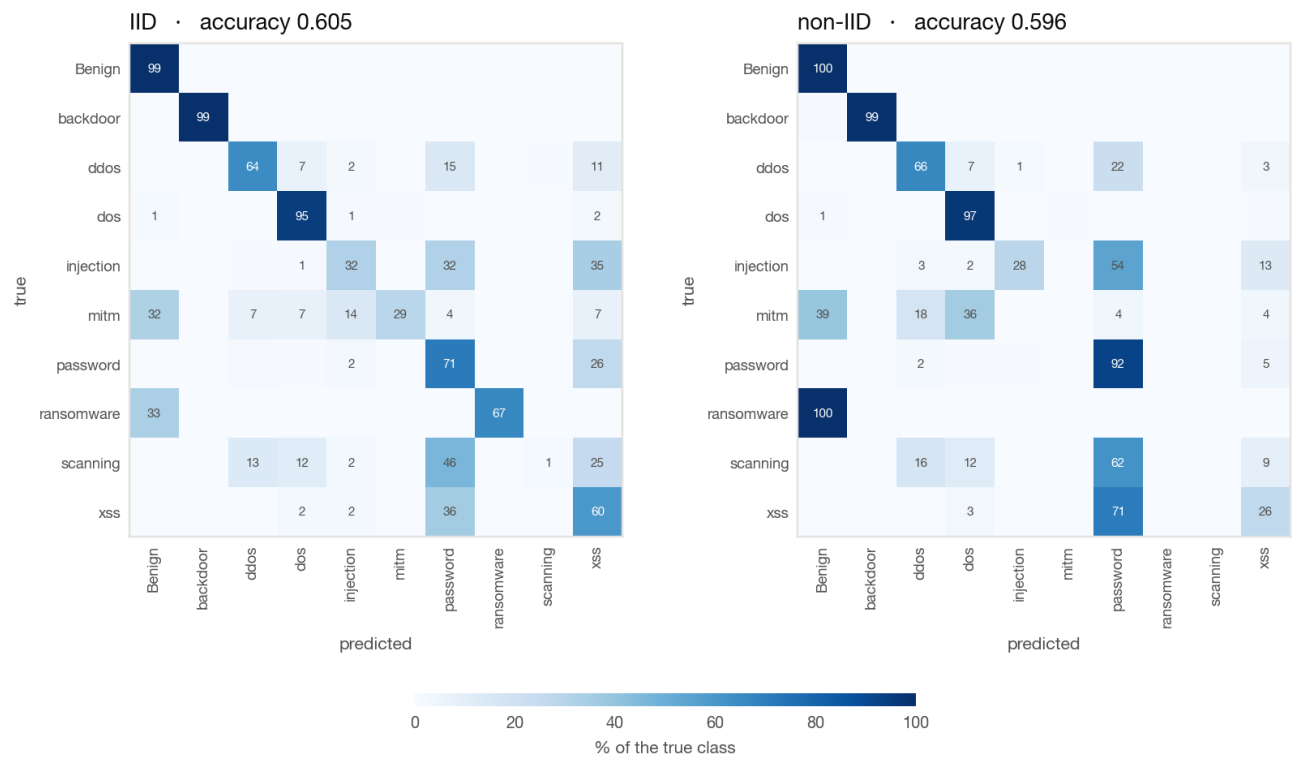
| Rare class | Non-IID shard split (full dataset)          | IID       |
|------------|---------------------------------------------|-----------|
| mitm       | C1 0 · C2 6 · <b>C3 745</b> · C4 6 · C5 280 | ~207 each |
| ransomware | C1 0 · C2 22 · C3 5 · <b>C4 86</b> · C5 1   | ~23 each  |

Under IID every client holds a share of these classes, so all five learn them and averaging reinforces what they learned. Under non-IID a single client holds most of the rows — 72 % of mitm sits on client 3 — and its update is averaged with four clients that never saw the class, which dilutes the signal to nothing.

The web-layer cluster is unmoved by the larger budget: injection 0.491 → 0.475, xss 0.283 → 0.297, scanning 0.011 → 0.027. Note also that ransomware has only 3 rows in the test set, so its F1 of 0.800 rests on two correct predictions.

# Confusion matrices — extended run

Rows are true classes, columns predicted; each cell is the percentage of that true class sent to that column.



Compared with the main study the diagonal is sharper on IID, including a mitm entry that was previously empty. The non-IID matrix keeps the mitm and ransomware rows blank.

## Status and next steps

| Run               | Status   | Training  | Wall clock | Communication |
|-------------------|----------|-----------|------------|---------------|
| ■ bert · IID      | complete | 10 rounds | 20 min     | 114 MB        |
| ■ bert · non-IID  | complete | 10 rounds | 20 min     | 114 MB        |
| ■ t5 · IID        | complete | 10 rounds | 35 min     | 338 MB        |
| ■ t5 · non-IID    | complete | 10 rounds | 35 min     | 338 MB        |
| ■ llama · IID     | complete | 10 rounds | 32 min     | 654 MB        |
| ■ llama · non-IID | complete | 10 rounds | 31 min     | 654 MB        |
| ■ qwen · IID      | complete | 10 rounds | 88 min     | 285 MB        |
| ■ qwen · non-IID  | complete | 10 rounds | 88 min     | 285 MB        |
| ■ gemma · IID     | complete | 10 rounds | 174 min    | 571 MB        |
| ■ gemma · non-IID | complete | 10 rounds | 173 min    | 571 MB        |

### Outstanding

1. **Exclude mitm and ransomware** and restate the headline numbers over eight classes. Both sit below 0.1 % of the data and the test set cannot support a claim either way; the GNN branch already excludes them.
2. **Centralised baselines**, to separate the cost of federation from the cost of the task itself.
3. **Aggregation study** — delta-SVD, FFA-LoRA and performance weighting. Worth measuring client divergence first: all clients start each round from the same broadcast and take only 63 steps, so if the adapters barely drift apart, naive FedAvg is already close to exact and the alternatives have little room to improve on it.
4. **More training budget**. Every curve is still rising at round 10 and each client has seen only 9 % of its own shard, so the models are underfit — this is the cheapest route to better numbers.
5. **Explain the gemma result**. It matches llama on IID and beats it under heterogeneity while being the smallest decoder here. Worth checking whether that holds at a second Dirichlet alpha before making anything of it.

### Open question

Absolute accuracy tops out near 0.60, and the per-class analysis points at why: four web-layer attacks are not separable from the ten flow fields alone. Graph structure — who talks to whom — is exactly the signal those fields are missing, so if the GNN branch separates that cluster, the comparison writes itself. Failing that, the defensible claim is the heterogeneity result: decoder models give up 4–8 % moving to non-IID where bert gives up 34 %.

### Reproducibility

Every run is seeded at 42 and logs its full dataset provenance to Comet: source file, row counts, Dirichlet alpha and per-client shard sizes. Rebuilding the shards from the same seed reproduces the partition byte for byte, so nothing needs to be carried between sessions.